Measuring DC Motor Parameters - Motor JGA25-370

Motivation

- There are a lot of applications that use the motor JGA25-370 controlled by Arduino Platform.
- In order to tune the control compensator, the plant transfer function is necessary, but there is insufficient data in motor datasheet to determine this.
- We can find a lot of parameter estimation and identification algorithms in the literature, which implies a complex implementation that requires a sofisticated lab structure.
- In this context, this page presents a simple proposal to obtain the DC motor parameters using a simple lab structure:
  - PC computer
  - Arduino UNO
  - DC Motor JGA25-370 with gearbox and encoder (Note: a second unit was used in one of the tests.)
  - L298N Motor Driver Module
  - 4x4 Keypad connected to HW-400 PCF8574 I2C Serial IO expansion board (Library available in github)
  - LCD with I2C Serial Interface Module (Library available in github).
  - Jumpers for connections
  - Hikari HX-120 LCR Meter
  - Hikari HM-2090 Digital Multimeter
  - DC Power Suppliers: 5V (LCD, keypad and interface modules - to not overload the voltage regulator in Arduino board), 12V (L298N bridge), USB or 9V* power source (to supply Arduino Board).

    * Recommended if some analog input will be used, since it provides a more stable voltage reference to ADC.
- It is important to emphasize that the intention here is to keep the method simple, as such a simple parameter estimation is obtained to support the motor control strategies. For better results, the reader should look into more precise identification algorithms, as well as better lab resources.
- Several dynamic aspects of the system were purposely left out in order to obtain only the parameters of the second-order dynamic model of a DC machine.

The Plant - Motor JGA25-370

- JGA25-370 is a brushed DC motor, where the magnetic field is generated by permanent magnets and possesses an integrated gearbox
- Fig.1 presents the JGA25-370 motor and the basic parameters available in the internet can be seen in Table I.

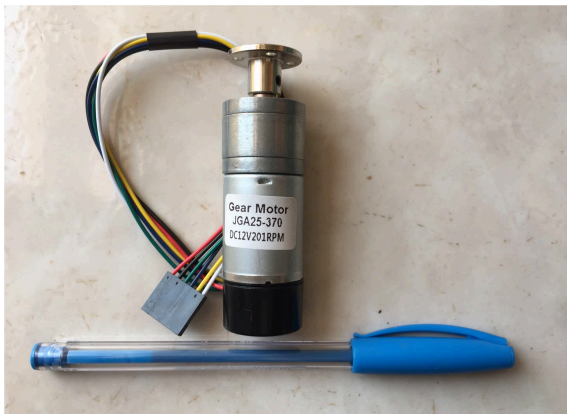


Fig.1 - Motor JGA25-370

Table I - JGA25-370 Datasheet Parameters

Quantity	Value	Unit		Quantity	Value	Unit
Nominal voltage	12	V		Gearbox relation	45:1	
No-load speed	188	rmp		Nominal speed	131	rpm
No-load current	50	mA		Nominal current	240	mA
Encoder relation	11	pulses/turn		Nominal torque	1.26	Kgf.cm

- Unfortunately, the data necessary to write the dynamic equations of JGA25-370 are not available in datasheet (Table I), so you can use the procedure presented below to get them.
- JGA25-370 is a small motor implying a low time constant. If the mechanical load related to the application has low inertia too, this can lead a very fast system, implying the necessity of a high sample rate to measure and control the system, as well as a powerfull microcontroller. As the idea here is to use a simple microcontroller, the system inertia moment was increased by coupling a rotating mass, as seen in Fig. 2

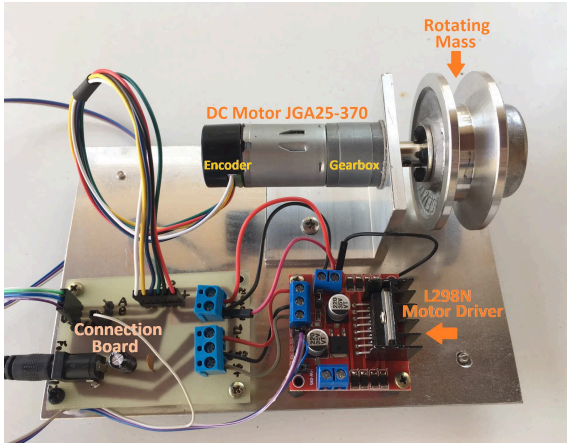


Fig.2 - Laboratory Prototype

Dynamic Model of a DC Motor

- In the present proposal, it will be considered the simple model of a DC motor. Dynamic aspects of the gearbox will be neglected.
- The scheme of the DC motor is shown in Fig.3.
- The loop equation of the electrical circuit of the armature (1) and the mechanical equation (2) of the motor are presented below, where:
  - *R* is the armature resistance;
  - *L* is the armature inductance;
  - *J* is the inertia moment;
  - *B* is the friction coefficient;
  - *e* is the back electromotive force;
  - V_a is the armature voltage;
  - T_m is the motor torque;
  - T_L is the load torque.

$$L\frac{di_a}{dt} + R\,i_a = V_a - e \quad (1)$$

$$J\frac{d\omega}{dt} + B\,\omega = T_m - T_L \quad (2)$$

- JGA25-370 doesn't have field winding and the magnetic field is produced by permanent magnets, then, for a constant magnetic field, the motor torque is proportional to the armature current, and the electromotive force is proportional to the speed, as determined by (3) and (4), respectively, where:
  - k_T is the torque constant;
  - k_e is back-emf constant.

$$T_m = k_T\,i_a \quad (3)$$

$$e = k_e\,\omega \quad (4)$$

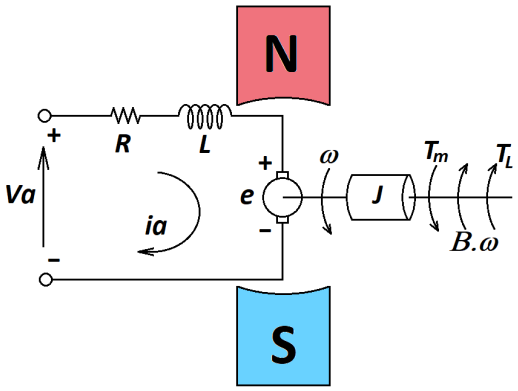


Fig.3 - DC Motor Scheme

- Considering the energy conversion, it is easy to see that $k_T = k_e$:

$$P_m = T_m\,\omega = P_e = e\,i_a \quad (5)$$

$$k_T\,i_a\,\omega = k_e\,\omega\,i_a \quad (6)$$

$$k_T = k_e = k \quad (7)$$

- Based on (1)-(4) and (7), it is possible to write the motor transfer function (8), which the respective diagram is presented in Fig.4.

$$\frac{\omega(s)}{V_a(s)} = \frac{k}{(Js+B)(Ls+R) + k^2} \quad (8)$$

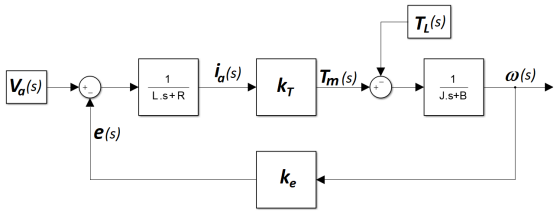


Fig.4 - DC Motor Transfer Function Diagram

Electrical Parameters of Armature

- The electrical parameters of armature were measured using the HIKARI HX-120 LCR digital meter:
  - Armaure resistance = 4.98 Ω
  - Armaure inductance = 3.8 mH

Torque and Back-EMF Constants

- Two JGA25-370 were coupled together. One of them was connected to L298N Bridge, controlled by the Arduino, in way that the speed could be varied. The armature voltage was measured in the second one. The results are presented below:

Table II - Speed and Armature Voltage Measurements

Speed (rps)	Speed (rad/s)	Armature Voltage (V)
2.07	13.0	7.5
1.82	11.44	6.55
1.52	9.55	5.51
1.26	7.92	4.4

- According to (4), the Table II data should represent a linear function crossing the origin, but considering the linear regression of these points, we can see some deviation. As it will be shown later, the lower the speed, the lower the speed measurement precision. Considering the highest speed, we have:

$$k_e = 0.577V/(rad/s) \quad (9)$$

- In order to determine the torque constant, the L298N bridge was connected to a 12V power supply, and the motor was driven with the maximum duty cycle, resulting in

V_a=10.6V, due to the saturation voltage over the bridge transistors, I_a=106mA and ω=2.78 rps (17.467 rad/s), which implies P=1.1236 W delivered to the motor.

- Subtracting the armature losses from the power delivered to the motor, we have:

$$P_m = P - R\, i_a^2 = 1.0676W \quad (10)$$

$$k_T = \frac{P_m}{\omega\, i_a} = 0.5766 N.m/A \quad (11)$$

- As it was expected, comparing (9) and (11), $k_e \cong k_T$.

Mechanical Parameters

- The system in Fig. 2 will be modeled according to the scheme presented in Fig. 4. The dynamic effect of the gearbox will be neglected, and it will be considered a unique inertia moment J, a friction coefficient B, and constant load torque T_L in the opposite direction of the rotation.
- The load torque (12) presents a non-linearity at the origin, as can be seen in Fig. 5. But in the following analysis, the torque can be assumed to be piecewise linear.

$$t_L = -signal(\omega) T_L \quad (12)$$

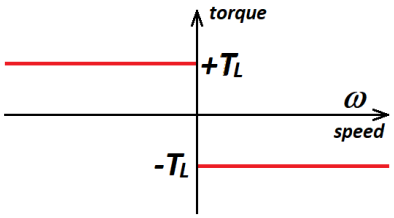


Fig.5 - Load Torque Characteristic

- Considering two distinct steady state points, where the acceleration torque is zero ($J\, d\omega/dt=0$), from (2), we have:

$$B\,\omega_1 + T_L = \frac{P_{m1}}{\omega_1} \quad (13)$$

$$B\,\omega_2 + T_L = \frac{P_{m2}}{\omega_2} \quad (14)$$

- Thus, from (13) and (14), B can be determined by (15). Replacing B in (14), as instance, we can find T_L.

$$B = \frac{\frac{P_{m2}}{\omega_2} - \frac{P_{m1}}{\omega_1}}{\omega_2 - \omega_1} \quad (15)$$

- Two tests were performed and the steady state data are presented in Table III.

Table III - Steady State Tests

Test 1		Test 2	

Armature Voltage V_{a1}	**7.19 V**	**Armature Voltage V_{a2}**	**12.1 V**
Armature Current I_{a1}	**0.0945 A**	**Armature Current I_{a2}**	**0.119 A**
Speed ω_1	**11.44 rad/s**	**Speed ω_2**	**19.67 rad/s**
Total Electric Power P_1	**0.6795 W**	**Total Electric Power P_2**	**1.4399 W**
Armature Losses P_{R1}	**0.0445 W**	**Armature Losses P_{R2}**	**0.0705 W**
Mechanical Power P_{m1}	**0.6350 W**	**Mechanical Power P_{m2}**	**1.3694 W**

- Considering Table III data and (15), *B* can be determined (16), as well T_L (17):

$$B = 0.00171 N.m/(rad/s) \quad (16)$$

$$T_L = 0.03593 N.m \quad (17)$$

- In order to determine the inertia moment *J*, it was considered the system deceleration curve, which is presented in Fig. 6. That is, the motor was accelerated, then when the speed achieved the steady state, the armature circuit was open. After that we have the motor deceleration until the speed reaches zero.
- As can be seen in Fig. 6, the derivative of speed presents a discontinuity when the speed reaches zero. This is due to the non-linearity of load torque shown in Fig. 5. To calculate *J* only the part corresponding to the linear response of the deceleration curve will be considered.
- Each blue circle in Fig.6 represents a speed sample. The initial speed is 2.58 rps. This implies 2.58*45=116.1 rps in the internal motor axis. In order to measure the speed the signals A and B of the encoder were connected to the pins INT0 and INT1 (external interrupts) of the Arduino UNO. These pins were programmed to request external interrupts at any logical change. So, this will implies 4 interrupt requests to each encoder pulse, or 44 interrupt requests per revolution. As a result, the interrupt counter incremented by both INT0 and INT1 ISR advances 5108 positions every second for a speed of 2.58 rps.
- The problem is that in a speed control loop for a fast system, it is not possible to wait one second to get the speed sample. This will implies a cost-benefit relation, since the high is the sample rate, the low is the resolution of the speed measurement. Considering a 100 Hz sample rate, the counter advances around 51 positions per period for a speed of 2.58 rps. This was the sample rate considered in the test presented in Fig.6, so we have one sample at each 0.01 s. The Arduino code used to get the results shown in Fig. 6 is presented below.

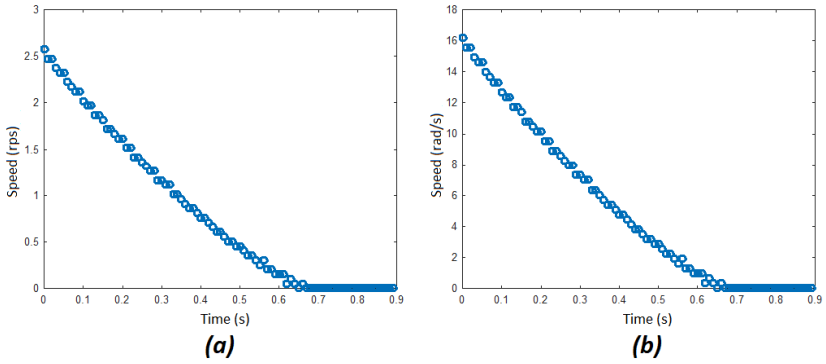


Fig.6 - Deceleration Curves: (a)rps, (b)rad/s

- Neglecting the load torque non-linearity and considering a constant torque T_L (like in a hoisting application) and the armature circuit open, that is, $T_m=0$, the system can be described by (17).
- Equation (17) is a non-homogeneous first order differential equation. We can use Laplace Transform to solve it. Thus, considering (18), we can determine the Laplace Transform of (17), as can be seen in (19).

$$J\frac{d\omega}{dt} + B\,\omega = -T_L \quad (17)$$

$$L\left\{\frac{d\omega(t)}{dt}\right\} = s\,L\{\omega(t)\} - \omega(0) \quad (18)$$

$$J\left[s\,\omega(s) - \omega_0\right] + B\,\omega(s) = -\frac{T_L}{s} \quad (19)$$

- After some algebraic manipulations, $\omega(s)$ can be determined (20).
- In order to determine the inverse Laplace Transform to get $\omega(t)$, it is necessary to apply the partial fraction decomposition on the right side of (20). The result is presented in (21).
- Then, determining the inverse Laplace Transform of (21), we have $\omega(t)$, as seen in (22).

$$\omega(s) = -\frac{T_L}{s(Js + B)} + \frac{J\,\omega_0}{Js + B} \quad (20)$$

$$\omega(s) = -\frac{T_L}{B\,s} - \frac{J\,T_L}{B(Js + B)} + \frac{J\,\omega_0}{Js + B} \quad (21)$$

$$\omega(t) = -\frac{T_L}{B} - \frac{T_L}{B}\,e^{-\frac{B}{J}t} + \omega_0\,e^{-\frac{B}{J}t} \quad (22)$$

- Considering the operation of the system shown in Fig. 2 in the speed range of $\omega\{0^+,\infty\}$, the system behavior can be described by (22) when $T_m=0$.
- In order to determine J, we can use an exponential regression tool. Note that B and T_L were determined previously by (16) and (17), respectively.
- As the exponential regression tools normally are related to basic functions, like that one presented in (23), we can apply a displacement in Y-axis of (22), as shown in (24), in order to fit in (23).

$$f(t) = a\, e^{-bt} \quad (23)$$

$$f(t) = \omega(t) + \frac{T_L}{B} = \left(\frac{T_L}{B} + \omega_0\right) e^{-\frac{B}{J}t} \quad (24)$$

- It is important to point out that only the part of the curve *ω (t)*, shown in Fig.6b, corresponding to the linear response will be considered for the purpose of the exponential regression.
- The result of exponential regression is presented in Fig. 7. Note that the graph's ordinate *ω (t)* has been shifted by $+T_L/B$.
- The exponential curve in Fig.7, according to the regression result, corresponds to (25).
- Then, considering (16), (24) and (25), we can determine J, as shown in (26).

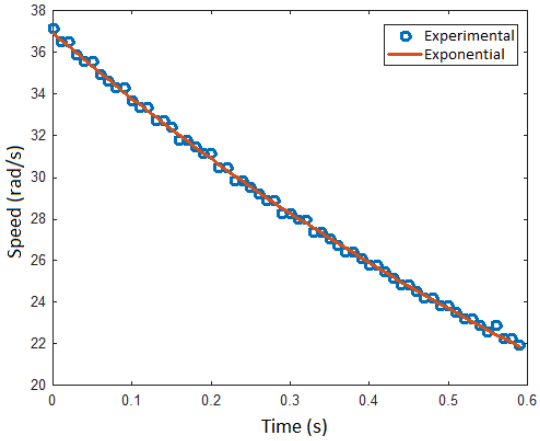


Fig.7 - Exponential Regression Result

$$f(t) = 36.9215e^{-0.88969\,t} \quad (25)$$

$$J = 0.0019258 \;\; Kg.m^2 \quad (26)$$

- A summary of the determined system data is presented in Table IV.

Table IV - System Parameter Summary - All units are in S.I.

Quantity	Value	Unit
Armature Resistance *R*	**4.98**	**Ω**
Armature Inductance *L*	**3.8**	**mH**
Inertia Moment *J*	**0.0019258**	**Kg.m^2**
Friction Coefficient *B*	**0.00171**	**N.m/(rad/s)**
Torque Constant k_T	**0.5766**	**N.m/A**
Back Electromotive Force Constant k_e	**0.577**	**V/(rad/s)**

The Plant Scheme

- The plant scheme used to get the presented results is shown in Fig.8. The Arduino code is presented below.

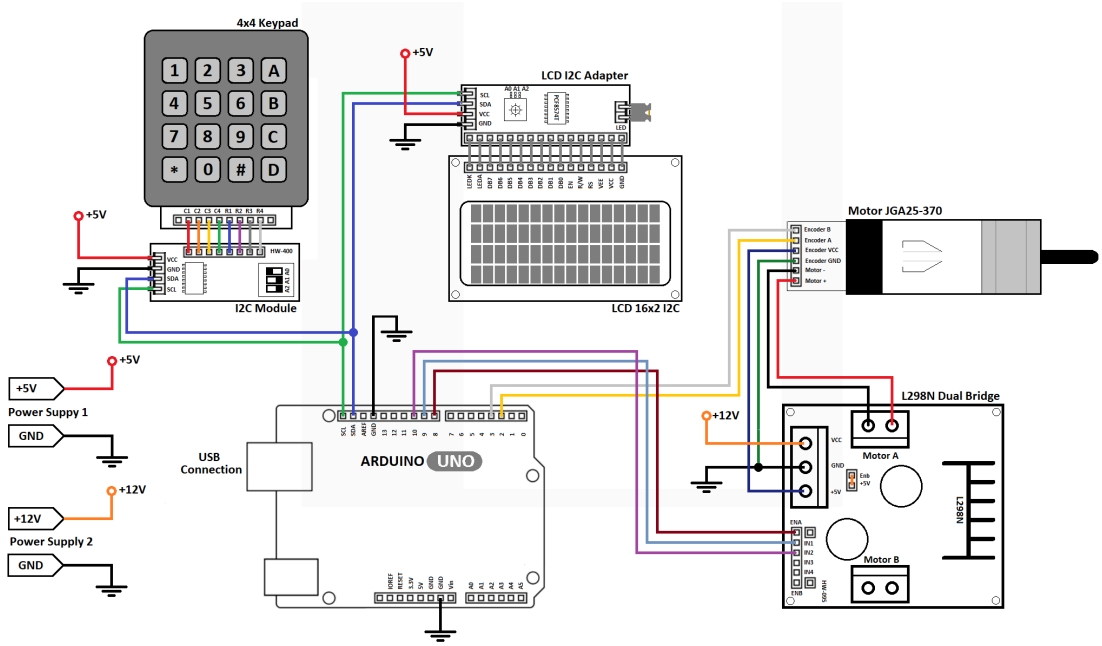


Fig.8 - The Plant Scheme

Quiz

- Based on the method presented in this page to get the parameters of JGA25-370 Motor, try to modify the Arduino C code below to provide the speed closed-loop control of the motor.
- Note that the plant transfer function is decribed by (8).
- In case to acquire electric quantities of the motor, see a way to use the ADC in the link below.
- Fig. 9 presents a comparison of simulation and experimental results, using the parameters obtained by the current method, considering a proportional-integral speed compensator with a not so aggressive adjustment.
- A more aggressive adjustment is presented in Fig. 10. In both cases the discrete PI presents an anti-windup scheme.
- **Hints**:
  - Atmega328p Timer 1 is a 16-bit counter, which provides a higher PWM resolution than Timer 0 or Timer2 (both have 8-bit).
  - To have low armature current ripple, increase the PWM carrier frequency (5kHz or higher). Using the unipolar PWM strategy, the switching frequency seen by the motor is twice the switching frequency of the bridge.
  - Unipolar PWM uses a triangular carrier and the bridge arms operate in a complementary way: *OCR1B = TOP - OCR1A*.
  - Keep in mind that in Arduino Programming, Timer 0 is used to generate the millisecond counter, if you use it to another purpose, the millisecond counter will be lost and the use of millis() function won't make sense.
  - Use the Timer 2 to generate the control sample rate. Based on a cpu clock of 16MHz, it is not possible to get 100Hz as a clock division result, but it is possible to get a near frequency, as 125Hz. It is also possible to configure, for example, 500 Hz and execute the control law every 5 interrupts. Keep in mind that the CPU must calculate the control action in a time interval significantly smaller than the

timer interrupt request rate for the real-time execution. As previously mentioned, the higher the sampling rate, the lower the accuracy of the speed measurement.

- To convert the control action from Volts to timer counting, consider (27), where *TOP* is the maximum counting value of the timer and V_{dc} is the DC link voltage of the bridge.

$$Timer\ Compare\ Value = \frac{Control\ Action}{Vdc}\ \frac{Top}{2} + \frac{Top}{2} \quad (27)$$

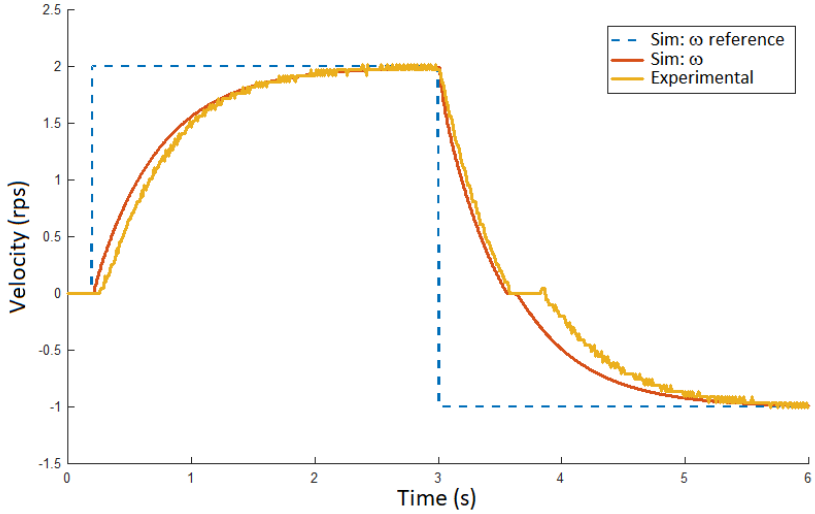


Fig.9 - Speed Control Results Using PI Compensator - Simulation and Experimental Comparison (Slow and robust adjustment)

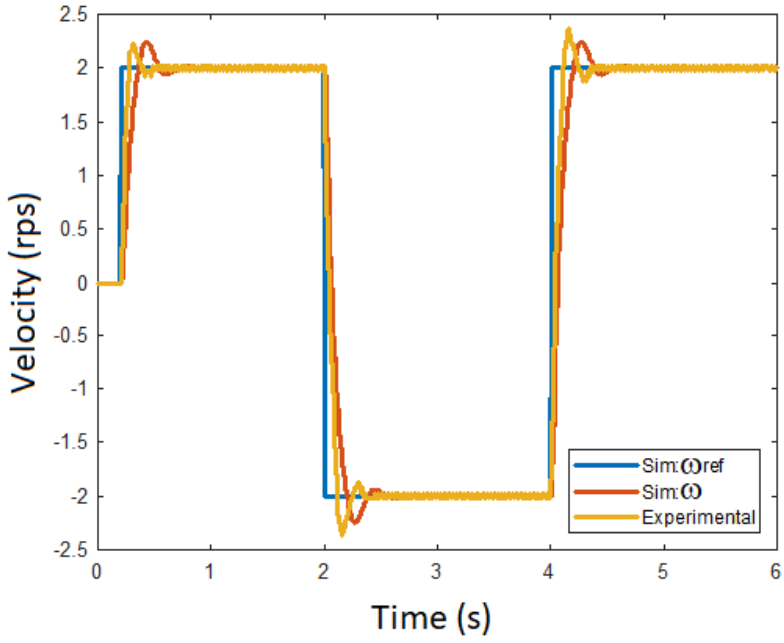


Fig.10 - Speed Control Results Using PI Compensator - Simulation and Experimental Comparison (More agressive adjustment)

Links

- Complete Arduino Code for Deceleration Curve Acquisition
- Arduino Code Example for Data Acquisition